

PLACEMENT PREPARATION NOTES

Practical Python & SQL Topics

A structured reference covering functional Python patterns, comprehensions, and conditional SQL logic — with runnable examples and quick-recall patterns for interviews.

filter() & map()

lambda

sorted()

dict unpacking

comprehensions

SQL CASE

TOPICS COVERED

13 core concepts

SECTIONS

Python · SQL

Python — Practical Placement Topics

1 filter() — Used to select elements based on a condition

```
even = list(filter(lambda x: x % 2 == 0, numbers))
```

filter() → **SELECT**

2 map() — Used to transform each element

```
double = list(map(lambda x: x * 2, numbers))
```

map() → **TRANSFORM**

3 lambda — Small anonymous function

```
lambda x: x * 2
```

Commonly used with:

```
map()  
filter()  
sorted()
```

4 sorted() — Used to sort data

```
employees = sorted(  
    employees,  
    key=lambda e: e["salary"],  
    reverse=True  
)
```

- `key` — tells Python what value to sort by
- `reverse=True` — descending order
- `reverse=False` — ascending order

5 Dictionary unpacking — **

```
{
    **employee,
    "salary": employee["salary"] * 1.10
}
```

****employee** preserves the existing dictionary fields, while the salary value is replaced with the new value.

6 List Comprehension

Basic:

```
squares = [x * x for x in numbers]
```

Equivalent to:

```
squares = []

for x in numbers:
    squares.append(x * x)
```

With condition:

```
even = [x for x in numbers if x % 2 == 0]
```

Transform + condition:

```
result = [x * 10 for x in numbers if x % 2 != 0]
```

Pattern:

```
[expression for item in iterable if condition]
```

7

Dictionary Comprehension

Pattern:

```
{key: value for item in iterable}
```

Example:

```
square = {x: x * x for x in numbers}
```

RESULT

```
{
  1: 1,
  2: 4,
  3: 9,
  4: 16,
  5: 25
}
```

With condition:

```
employees_dict = {
    employee["name"]: employee["salary"]
    for employee in employees
    if employee["salary"] >= 80000
}
```

RESULT

```
{
  "Alice": 100000,
  "Bob": 80000
}
```

8

Set Comprehension

```
numbers = [1, 2, 2, 3, 3, 4]

unique_squares = {x * x for x in numbers}
```

RESULT

```
{1, 4, 9, 16}
```

Because sets automatically remove duplicates.

9

Nested Comprehension

Given:

```
matrix = [  
    [1, 2],  
    [3, 4]  
]
```

Flatten it:

```
result = [x for row in matrix for x in row]
```

RESULT

```
[1, 2, 3, 4]
```

With condition:

```
even = [x for row in matrix for x in row if x % 2 == 0]
```

RESULT

```
[2, 4]
```

Equivalent normal loop:

```
result = []  
  
for row in matrix:  
    for x in row:  
        if x % 2 == 0:  
            result.append(x)
```

Important understanding: In `[x for row in matrix for x in row]`, **row** is the actual row list, not its index. And **x** is the actual element, not its index.

SQL — CASE

10 Basic CASE

CASE is SQL's conditional logic, similar to if/else.

```
SELECT name,  
       salary,  
       CASE  
         WHEN salary >= 90000 THEN 'High'  
         WHEN salary >= 70000 THEN 'Medium'  
         ELSE 'Low'  
       END AS category  
FROM Employee_1;
```

Important: CASE checks conditions from top to bottom. The first true condition is used.

11 CASE with calculations

```
SELECT name,  
       salary,  
       CASE  
         WHEN salary >= 80000 THEN salary * 0.10  
         ELSE salary * 0.05  
       END AS bonus  
FROM Employee_1;
```

Here CASE is not just producing text — it can perform calculations.

12 CASE + GROUP BY

```
SELECT COUNT(*),  
       CASE  
         WHEN salary >= 90000 THEN 'High'  
         WHEN salary >= 70000 THEN 'Medium'  
         ELSE 'Low'  
       END AS category  
FROM Employee_1  
GROUP BY category;
```

Conceptually:

```
CASE      → creates categories  
GROUP BY  → groups them  
COUNT(*) → counts each group
```

13 Conditional COUNT()

Very important interview pattern:

```
SELECT COUNT(  
  CASE  
    WHEN salary >= 80000 THEN 1  
  END  
) AS employees_earning_80000_plus  
FROM Employee_1;
```

Why?

```
salary >= 80000 → 1  
otherwise       → NULL
```

COUNT() counts non-NULL values.

This effectively means: Count rows satisfying a condition.